PRACTICAL - 08

Aim: Guarded calculator (DivideByZeroException.java + Driver.java)

Code Screenshot:

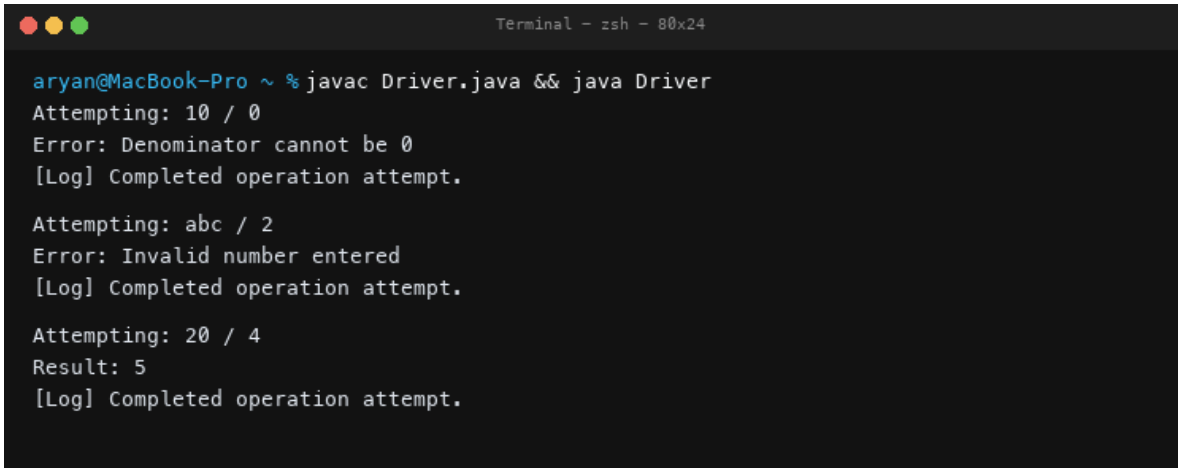
DivideByZeroException.java

```java
public class DivideByZeroException extends Exception {
    public DivideByZeroException(String msg) {
        super(msg);
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        String[] inputs = { "10 / 0", "abc / 2", "20 / 4" };
        for (String input : inputs) {
            System.out.println("Attempting: " + input);
            try {
                String[] parts = input.split(" ");
                int a = Integer.parseInt(parts[0]);
                int b = Integer.parseInt(parts[2]);
                if (b == 0) throw new DivideByZeroException("Denominator cannot be 0");
                System.out.println("Result: " + (a / b));
                break;
            } catch (DivideByZeroException e) {
                System.out.println("Error: " + e.getMessage());
            } catch (NumberFormatException e) {
                System.out.println("Error: Invalid number entered");
            } finally {
                System.out.println("[Log] Completed operation attempt\n");
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to create custom checked exceptions extending Exception.
- Mastered multi-catch blocks for differentiated error handling.
- Understood the execution guarantees of the finally block.
- Implemented resilient retry loops for user-driven input.
- Avoided abrupt program termination through structured exception handling.

Aim: Stock issue (OutOfStockException.java + Driver.java)

Code Screenshot:

OutOfStockException.java

```java
public class OutOfStockException extends Exception {
    private int shortfall;
    public OutOfStockException(String item, int shortfall) {
        super("Out of stock for " + item + " (short by " + shortfall + ")");
        this.shortfall = shortfall;
    }
    public int getShortfall() { return shortfall; }
}

class Warehouse {
    private int stock = 10;
    public void issue(String item, int qty) throws OutOfStockException {
        if (qty <= 0) throw new IllegalArgumentException("Quantity must be > 0");
        if (qty > stock) throw new OutOfStockException(item, qty - stock);
        stock -= qty;
        System.out.println("Issued " + qty + " of " + item + ". Remaining: " + stock);
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Warehouse wh = new Warehouse();
        int[] orders = { 4, -2, 8, 3 };
        for (int qty : orders) {
            try {
                wh.issue("Laptop", qty);
            } catch (OutOfStockException e) {
                System.out.println("Stock Failure: " + e.getMessage() + " [Shortfall: " + e.getShortfall() + "]");
            } catch (IllegalArgumentException e) {
                System.out.println("Invalid Request: " + e.getMessage());
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to design exceptions that carry rich contextual error state.
- Understood checked vs unchecked exceptions and when to use each.
- Implemented fault-tolerant batch processing over item requests.
- Learned proper exception propagation and handling boundaries.
- Gained insights into enterprise inventory management error strategies.